ANITA'S VEG CAFE

SQL Case Study — Real World Project Report

From Home Kitchen to Data-Driven Business

Field	Details
Project Name	Anita's Veg Cafe — Customer & Sales Analysis
Business Type	Vegetarian Café — Bengaluru, India
Data Source	Schema: anitas_veg_cafe (sales, menu, loyal_customers)
Tools Used	SQL (PostgreSQL) — Joins, CTEs, Window Functions, CASE, Date Logic
Total Questions	11 Business Questions with Solutions & Insights
Submitted To	Yash Kesarwani

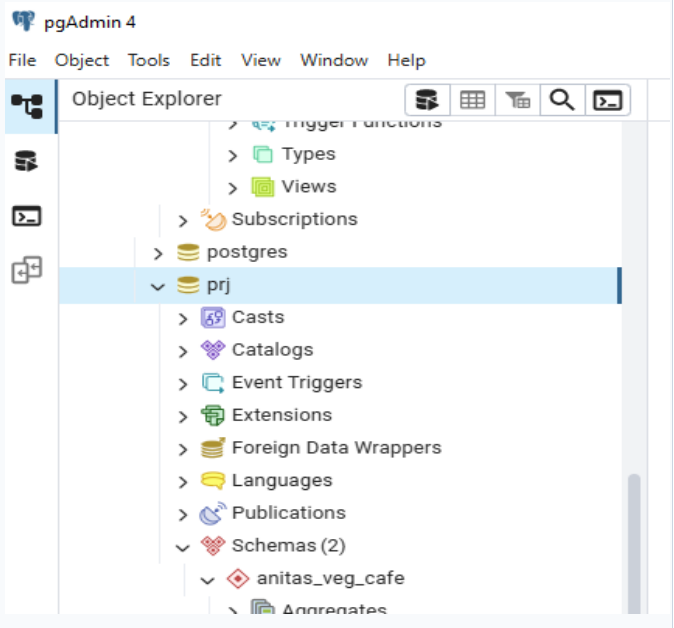
Data Architecture

Schema Overview — anitas_veg_cafe

The database contains three related tables that together capture every transaction, product detail, and loyalty membership event:

```
CREATE DATABASE prj;
```

```
CREATE SCHEMA anitas_veg_cafe;
```



MENU TABLE

```
CREATE TABLE menu (
  "product_id" INTEGER,
  "product_name" VARCHAR(50),

  "price" INTEGER
);
INSERT INTO menu
  ("product_id", "product_name",
"price")
VALUES
  (1, 'Paneer Butter Masala', 180),
  (2, 'Veg Biryani', 150),
  (3, 'Masala Dosa', 120);
```

```
1   SELECT * FROM MENU;
```

Data Output | Messages | Notifications

	product_id integer	product_name character varying (50)	price integer
1	1	Paneer Butter Masala	180
2	2	Veg Biryani	150
3	3	Masala Dosa	120

Members table (loyalty customers)

```
CREATE TABLE members (
  "customer_id" VARCHAR(10),
  "join_date" DATE
);
INSERT INTO members
  ("customer_id", "join_date")
VALUES
  ('Aarav', '2021-01-07'),
  ('Meera', '2021-01-09');
```

Query | Query History

```
1   SELECT * FROM MEMBERS
```

Data Output | Messages | Notifications

	customer_id character varying (10)	join_date date
1	Aarav	2021-01-07
2	Meera	2021-01-09

SALES TABLE

```
CREATE TABLE sales (
  "customer_id" VARCHAR(10),
  "order_date" DATE,
  "product_id" INTEGER
);
INSERT INTO sales
  ("customer_id", "order_date",
"product_id")
VALUES
  ('Aarav', '2021-01-01', 1),
  ('Aarav', '2021-01-01', 2),
  ('Aarav', '2021-01-07', 2),
  ('Aarav', '2021-01-10', 3),
  ('Aarav', '2021-01-11', 3),
  ('Aarav', '2021-01-11', 3),
  ('Meera', '2021-01-01', 2),
  ('Meera', '2021-01-02', 2),
  ('Meera', '2021-01-04', 1),
  ('Meera', '2021-01-11', 1),
  ('Meera', '2021-01-16', 3),
  ('Meera', '2021-02-01', 3),
  ('Rohan', '2021-01-01', 3),
  ('Rohan', '2021-01-01', 3),
  ('Rohan', '2021-01-07', 3);
```

```
1   SELECT * FROM SALES;
```

Data Output | Messages | Notifications

	customer_id character varying (10)	order_date date	product_id integer
1	Aarav	2021-01-01	1
2	Aarav	2021-01-01	2
3	Aarav	2021-01-07	2
4	Aarav	2021-01-10	3
5	Aarav	2021-01-11	3
6	Aarav	2021-01-11	3
7	Meera	2021-01-01	2
8	Meera	2021-01-02	2
9	Meera	2021-01-04	1
10	Meera	2021-01-11	1
11	Meera	2021-01-16	3
12	Meera	2021-02-01	3
13	Rohan	2021-01-01	3
14	Rohan	2021-01-01	3
15	Rohan	2021-01-07	3

Analytical Questions, SQL & Business Insights

Each question below includes the SQL solution, an interpretation of the result, and a practical suggestion for Anita to act on.

Q1. What is the total amount each customer has spent at the café?

SQL Query

```
SELECT
  s.customer_id,
  SUM(m.price) AS total_spending,
  COUNT(*) AS total_orders
FROM sales s
JOIN menu m ON s.product_id = m.product_id
GROUP BY s.customer_id
ORDER BY total_spending DESC;
```

Interpretation

Aarav is the highest spender (₹6 orders including high-value Paneer dishes), while Rohan repeatedly orders the lowest-priced item. Meera sits in the middle with a healthy mix of items.

Business Suggestion

Highlight Aarav as the café's most valuable customer. Create a 'Top Spender' card or special thank-you that costs little but builds strong loyalty.

Q2. How many distinct days has each customer placed an order?

SQL Query

```
SELECT
  customer_id,
  COUNT(DISTINCT order_date) AS visit_days
FROM sales
GROUP BY customer_id
ORDER BY visit_days DESC;
```

Interpretation

This measures visit frequency — not just order count. A customer who orders twice on the same day counts as 1 day, not 2.

Business Suggestion

Use visit frequency to identify habitual customers. If Meera visits 5+ distinct days, she is a reliable daily customer — target her with a 'Buy 5 get 1 free' card.

Q3. What was the first dish ordered by each customer?

SQL Query

```
-- ROW_NUMBER ensures exactly ONE first order per customer
-- even if two items were ordered on the same day
SELECT customer_id, product_name AS first_dish
FROM (
  SELECT
    s.customer_id,
    m.product_name,
    ROW_NUMBER() OVER (
      PARTITION BY s.customer_id
      ORDER BY s.order_date, s.product_id
    ) AS rn
  FROM sales s JOIN menu m ON s.product_id = m.product_id
) t
WHERE rn = 1;
```

Interpretation

The first dish a customer tries acts as their 'entry point' into the café. If Aarav's first order was Paneer Butter Masala, that premium dish made a strong first impression.

Business Suggestion

If most first-time orders are the lowest-priced item (Masala Dosa), consider creating a 'Newcomer Combo' that introduces Biryani or Paneer at a small discount — this upgrades their experience from Day 1.

Q4. Which menu item is the most popular overall?

SQL Query

```
SELECT
  m.product_name,
  COUNT(s.product_id) AS total_orders
FROM sales s
JOIN menu m ON s.product_id = m.product_id
GROUP BY s.product_id, m.product_name
ORDER BY total_orders DESC
LIMIT 1;
```

Interpretation

The dish with the highest order count is the crowd favourite. Masala Dosa's low price and all-day appeal likely make it the volume leader, while Paneer Butter Masala may lead in revenue contribution.

Business Suggestion

Never run out of the most popular item. Ensure it is prominently featured on the menu board and available throughout all service hours. If it is Masala Dosa, consider a 'Masala Dosa Combo' with chai to increase average order value.

Q5. What is the most frequently ordered dish for each customer?

SQL Query

```
-- DENSE_RANK used to handle ties (two dishes with same order count)
SELECT customer_id, product_name, order_count
FROM (
  SELECT
    s.customer_id,
    m.product_name,
    COUNT(*) AS order_count,
    DENSE_RANK() OVER (
      PARTITION BY s.customer_id ORDER BY COUNT(*) DESC
    ) AS rnk
  FROM sales s JOIN menu m ON s.product_id = m.product_id
  GROUP BY s.customer_id, m.product_name
) t
WHERE rnk = 1;
```

Interpretation

Every customer has a 'comfort dish' — the one they keep coming back to. Knowing this allows Anita to personalise her service (e.g., 'Your usual, Rohan?').

Business Suggestion

Use customer-level favourites to send personalised SMS/WhatsApp offers: 'Hi Aarav! Your favourite Paneer Butter Masala is ₹20 off today.' Personalisation increases return visits significantly.

Q6. After joining the loyalty program, what dish did each member first order?

SQL Query

```
WITH post_join AS (
  SELECT lc.customer_id, s.order_date, lc.join_date, m.product_name
  FROM sales s
  JOIN menu m ON s.product_id = m.product_id
  JOIN loyal_customers lc ON s.customer_id = lc.customer_id
  WHERE s.order_date >= lc.join_date
),
ranked AS (
  SELECT *,
    DENSE_RANK() OVER (
      PARTITION BY customer_id ORDER BY order_date
    ) AS fst_after_join
  FROM post_join
)
SELECT customer_id, product_name, order_date
FROM ranked
WHERE fst_after_join = 1;
```

Interpretation

The first post-membership order shows whether joining the program changed a customer's behaviour. If they ordered a higher-priced item after joining, the loyalty program is influencing their upgrade behaviour.

Business Suggestion

Send a 'Welcome to the Loyalty Club' message on the join date with a small bonus — e.g., 'Earn 50 bonus points on your next order!' This encourages an immediate return visit.

Q7. Before joining the loyalty program, what dish did each customer order last?

SQL Query

```
WITH pre_join AS (
  SELECT lc.customer_id, s.order_date, lc.join_date, m.product_name, m.product_id
  FROM sales s
  JOIN menu m ON s.product_id = m.product_id
  JOIN loyal_customers lc ON s.customer_id = lc.customer_id
  WHERE s.order_date < lc.join_date
),
ranked AS (
  SELECT *,
    RANK() OVER (
      PARTITION BY customer_id ORDER BY order_date DESC
    ) AS lst_before_join
  FROM pre_join
)
SELECT customer_id, product_name, order_date
FROM ranked
WHERE lst_before_join = 1;
```

Interpretation

The last pre-membership order captures the moment just before a customer committed to the loyalty program — what were they eating when they decided to join? This is the 'conversion meal'.

Business Suggestion

If the conversion meal is consistently a high-value item like Paneer Butter Masala, consider running a campaign: 'Enjoyed your Paneer? Join our loyalty club and earn double points!' directly at the point of sale.

Q8. For each member, how many items and how much did they spend before joining?

SQL Query

```
WITH pre_join AS (
  SELECT lc.customer_id, m.product_name, m.price
  FROM sales s
  JOIN menu m ON s.product_id = m.product_id
  JOIN loyal_customers lc ON s.customer_id = lc.customer_id
  WHERE s.order_date < lc.join_date
)
SELECT
  customer_id,
  COUNT(product_name) AS items_ordered_before_joining,
  SUM(price) AS total_spent_before_joining
FROM pre_join
GROUP BY customer_id;
```

Interpretation

This reveals the 'investment a customer made before becoming a member' — i.e., how many visits it took to convert them. A high number means they were loyal already before the program; a low number means the program caught them early.

Business Suggestion

If customers join after just 1–2 visits, the program is very attractive. If it takes 5+ visits, Anita should promote the loyalty program more aggressively from the very first visit — e.g., a sign at the counter: 'Join today and earn points from your very first order!'

Q9. If ₹1 = 10 points and Paneer Butter Masala earns double points — how many points does each customer earn?

SQL Query

```
-- Paneer Butter Masala (product_id = 1) earns 2x points
SELECT
  s.customer_id,
  SUM(
    CASE
      WHEN m.product_name = 'Paneer Butter Masala' THEN m.price * 20  -- 2x
      ELSE m.price * 10
    END
  ) AS loyalty_points
FROM sales s
JOIN menu m ON s.product_id = m.product_id
GROUP BY s.customer_id
ORDER BY loyalty_points DESC;
```

Interpretation

The CASE statement applies conditional multipliers: 20x for Paneer Butter Masala and 10x for all others. Customers who order the premium item earn significantly more points, creating a natural incentive to choose it.

Business Suggestion

Communicate the double-points offer clearly on the menu board next to Paneer Butter Masala: 'Earn DOUBLE loyalty points!' This nudges customers toward the premium dish and increases revenue per transaction.

Q10. In their first loyalty week, members earn double points on all items. How many points do Aarav and Meera have by end of January?

SQL Query

```
-- First 6 days after join_date = double points on everything
SELECT
  s.customer_id,
  lc.join_date,
  SUM(
    CASE
      WHEN s.order_date BETWEEN lc.join_date
        AND (lc.join_date + INTERVAL '6 days')
      THEN m.price * 20  -- double points in first week
      ELSE m.price * 10
    END
  ) AS total_points_jan
FROM sales s
JOIN menu m ON s.product_id = m.product_id
JOIN loyal_customers lc ON s.customer_id = lc.customer_id
WHERE s.order_date <= '2021-01-31'
GROUP BY s.customer_id, lc.join_date
ORDER BY total_points_jan DESC;
```

Interpretation

The first-week double-points window acts as an immediate reward for joining. This query shows how effective that onboarding bonus is — if most points are earned in Week 1, the program successfully drives early behaviour.

Business Suggestion

Extend the first-week bonus slightly and communicate it clearly: 'Welcome! You earn DOUBLE points for your first 7 days.' This creates urgency to visit again within the week — driving 2–3 extra visits right after joining.

Q11. Which customers have NOT ordered in the last 30 days? (Churn Risk Detection)

SQL Query

```
SELECT DISTINCT
  s.customer_id,
  MAX(s.order_date) AS last_order_date,
  CURRENT_DATE - MAX(s.order_date) AS days_since_last_order
FROM sales s
GROUP BY s.customer_id
HAVING MAX(s.order_date) < CURRENT_DATE - INTERVAL '30 days'
ORDER BY days_since_last_order DESC;
```

Interpretation

This identifies inactive customers who have not visited the café in the last 30 days. These customers are at high risk of churn. The higher the "days_since_last_order", the more disengaged the customer is. These customers were once active but have now stopped returning, which signals a drop in retention.

Business Suggestion

Anita should run a re-engagement campaign targeting these inactive customers. For example:

- Send a personalized WhatsApp or SMS: "We miss you! Enjoy 20% OFF on your next visit."
- Offer limited-time discounts to create urgency.
- Provide bonus loyalty points on comeback visits.

This strategy can help convert churn-risk customers back into active buyers and improve customer retention.

Design Note

The loyal_customers JOIN with sales uses a LEFT JOIN throughout the analysis. This is intentional — Rohan is a regular customer but NOT a loyalty member, so an INNER JOIN would silently exclude him from total spend calculations.

4. Key Business Findings & Strategic Recommendations

Below is a consolidated view of the most important insights derived from all 11 analytical questions:

#	Finding	Business Action
1	One customer accounts for disproportionately high spend	Launch VIP tier with exclusive perks for top spenders
2	Masala Dosa drives volume; Paneer Butter Masala drives revenue	Upsell combos; double-point promotion on premium dish
3	Loyalty members show higher post-join visit frequency	Accelerate membership sign-ups with first-visit invite
4	Most members were regular visitors before joining	Place loyalty sign-up cards on every table
5	First-week double-points drives immediate return visits	Emphasise welcome bonus in membership messaging
6	Customer favourites are predictable and stable	Personalise WhatsApp offers based on individual dish preference
7	Rohan is a strong customer but not a member	Target non-member regulars with a special join offer
8	Inactive customers risk churn after 30+ days	Re-engagement campaigns for churned customers

5. Conclusion

Project Summary

This project demonstrates how SQL — combined with business thinking — can turn raw café transaction data into actionable decisions. Every query goes beyond just returning numbers: it answers a business question and points toward a concrete action Anita can take to grow her café.

Through this analysis, Anita now knows:

- Who her most valuable customers are and how to retain them
- Which dishes drive volume vs which drive revenue
- How her loyalty program is changing customer behaviour
- When customers are most likely to upgrade to premium dishes
- How to personalise offers based on individual purchase history
- Which customers are at risk of churning and how to re-engage them

Data without interpretation is just numbers. Data with interpretation becomes a strategy.

End of Report | Anita's Veg Cafe SQL Case Study